

UNAM

Universidad Nacional Autónoma de México

Facultad de Ciencias

Modelado y Programación

Pong Evolved

Equipo Polimórfo

Juárez Larrondo Omar Alejandro
(322245244)

Soto Mendoza Ismael de Jesús
(322210422)

Vega Navas Saúl (322088267)

Problemática

El desarrollo de aplicaciones interactivas presenta desafíos significativos en términos de arquitectura de software, particularmente cuando se requiere mantener la flexibilidad y extensibilidad del sistema a lo largo de su ciclo de vida. La problemática central abordada en este proyecto radica en la complejidad inherente a la construcción de sistemas donde múltiples componentes deben colaborar de manera cohesiva sin introducir acoplamiento excesivo entre ellos. En el contexto de aplicaciones de entretenimiento digital, como los videojuegos, esta complejidad se amplifica debido a la necesidad de gestionar estados dinámicos, interacciones en tiempo real, y la coordinación entre lógica de negocio, presentación visual y control de entrada del usuario.

Los sistemas que no implementan una separación clara de responsabilidades tienden a sufrir de rigidez, manifestándose en la dificultad para introducir cambios sin afectar múltiples componentes del sistema. Asimismo, la fragilidad se presenta cuando modificaciones en una parte del código producen efectos secundarios inesperados en módulos aparentemente no relacionados. La inmovilidad impide la reutilización de componentes en otros contextos, mientras que la viscosidad dificulta la implementación de mejoras al hacer que las soluciones incorrectas sean más fáciles de implementar que las correctas. Estos problemas son particularmente evidentes en aplicaciones que requieren extensibilidad, como aquellas que deben soportar múltiples modos de operación, niveles de dificultad variables, o funcionalidades de personalización.

El alcance de este proyecto consiste en desarrollar una reimplementación moderna del clásico juego Pong utilizando JavaFX como framework de interfaz gráfica, demostrando la aplicación práctica de catorce patrones de diseño diferentes en un total de veintitrés instancias a lo largo de la arquitectura del sistema. La aplicación implementa tres modos de juego distintos: un modo de un jugador contra inteligencia artificial con tres niveles de dificultad configurables, un modo multijugador local para dos jugadores, y un editor de niveles personalizados que permite a los usuarios crear y persistir sus propias configuraciones de juego. El sistema incorpora un motor de inteligencia artificial adaptativo, un subsistema de persistencia basado en SQLite para el almacenamiento de niveles personalizados, y una arquitectura modular que facilita la extensión y mantenimiento del código. La implementación se basa en el patrón arquitectónico Modelo-Vista-Controlador como estructura fundamental, complementado con trece patrones adicionales que abordan problemáticas específicas del dominio de aplicación.

La justificación de la solución propuesta se fundamenta en la aplicación sistemática de principios de diseño orientado a objetos y patrones arquitectónicos probados que mitigan los problemas anteriormente descritos. La arquitectura Modelo-Vista-Controlador establece una separación fundamental entre la lógica de negocio, la presentación visual y el control de flujo, previniendo el acoplamiento entre estos tres aspectos críticos del sistema. El patrón Strategy permite encapsular algoritmos intercambiables, facilitando la implementación de diferentes niveles de dificultad de la inteligencia artificial y distintas estrategias de colisión sin modificar el código cliente. El patrón Observer desacopla la lógica del juego de sus efectos secundarios, como la reproducción de audio o la actualización de la interfaz visual, permitiendo que estos subsistemas evolucionen independientemente. Los patrones Factory y Builder abstraen la complejidad de creación de objetos complejos, mientras que Singleton garantiza el acceso controlado a recursos compartidos como la configuración global del sistema. El patrón Memento permite la implementación de funcionalidad de deshacer en el editor sin exponer la estructura interna de los objetos, mientras que Adapter proporciona una interfaz unificada para diferentes esquemas de control de entrada. El patrón Composite facilita el tratamiento uniforme de objetos individuales y compuestos en la jerarquía de entidades del juego, y el patrón Prototype permite la clonación eficiente de configuraciones de paletas. Adicionalmente, se emplean los patrones Repository y Data Transfer Object para abstraer y optimizar el acceso a datos persistentes, Facade para simplificar interfaces complejas de subsistemas, y Value Object para representar conceptos inmutables del dominio. Esta arquitectura basada en patrones no solo previene los problemas de rigidez, fragilidad, inmovilidad y viscosidad, sino que además proporciona un sistema altamente mantenible, extensible y comprensible que facilita la evolución futura del software y sirve como demostración práctica de principios fundamentales de ingeniería de software.

Patrones de Diseño Utilizados

El proyecto implementa una arquitectura robusta fundamentada en el patrón **Model-View-Controller (MVC)**, el cual constituye la columna vertebral de la aplicación. Este patrón arquitectónico separa la lógica de negocio del modelo, implementada en `ModeloJuego.java` y las clases de entidades ubicadas en `mvc.modelo.entidades`, de la representación visual gestionada por las vistas (`VistaMenu.java`, `VistaJuego.java`, `VistaEditor.java`, `VistaSeleccionDificultad.java`, `VistaSeleccionNiveles.java` y `GestorEscenas.java`), mientras que los controladores (`ControladorBase.java`, `ControladorMenu.java`, `ControladorJuego.java`, `ControladorEditor.java`, `ControladorSeleccionDificultad.java`, `ControladorSeleccionNiveles.java`, `ControladorDialogoCargarNivel.java` y `ControladorDialogoSeleccionPorDificultad.java`) coordinan la comunicación bidireccional entre ambas capas. La adopción de MVC resulta fundamental para mantener un código organizado, facilitar el mantenimiento y permitir modificaciones independientes en la interfaz gráfica sin afectar la lógica del juego.

El patrón **Strategy** se encuentra presente en tres contextos distintos del sistema. En primer lugar, las estrategias de movimiento permiten encapsular diferentes algoritmos de desplazamiento para las paletas, diferenciando entre el movimiento controlado por el jugador y el comportamiento de la inteligencia artificial. La interfaz `EstrategiaMovimiento.java` define el contrato que implementan `EstrategiaMovimientoJugador.java` y `EstrategiaMovimientoIA.java`, esta última auxiliada por componentes especializados como `GeneradorErrorMovimiento.java`, `TemporizadorReaccion.java` y `CalcularTrayectoria.java`. Esta estructura facilita la modificación de comportamientos sin alterar el código de las paletas y permite ajustar dinámicamente la dificultad de la IA durante la ejecución.

En segundo lugar, las estrategias de colisión gestionan las interacciones físicas entre diferentes elementos del juego mediante la interfaz `EstrategiaColision.java` y sus implementaciones especializadas: `EstrategiaColisionPelotaPaleta.java`, `EstrategiaColisionPelotaBloque.java` y `EstrategiaColisionPelotaPared.java`, coordinadas por `GestorColisiones.java`. Esta separación permite que cada tipo de colisión tenga su propia lógica específica, simplificando la extensión del sistema con nuevos tipos de objetos colisionables. Finalmente, las estrategias de items representan diferentes efectos aplicables mediante power-ups, donde la interfaz `Item.java` es implementada por `ItemAumentoVelocidad.java`, `ItemNeblina.java` e `ItemRedimensionarPaleta.java`, proporcionando un mecanismo extensible para incorporar nuevas mecánicas de juego sin modificar el código existente.

El patrón **Observer** establece un sistema de notificaciones que permite a diversos componentes reaccionar ante eventos del juego sin acoplamiento directo. La interfaz `ObservadorJuego.java` define los métodos de notificación que implementan `GestorAudio.java` y `ObservadorUI.java`, permitiendo que el sistema de audio y la interfaz visual respondan automáticamente a eventos como colisiones o cambios de puntuación. Este diseño facilita la adición de nuevos observadores sin modificar el modelo del juego, manteniendo una separación clara de responsabilidades.

El patrón **Singleton** garantiza la existencia de una única instancia para componentes críticos del sistema. `ConfiguracionGlobal.java` implementa el patrón mediante inicialización perezosa tradicional, centralizando configuraciones globales como el modo de pantalla completa. `GestorPrototiposPaleta.java` utiliza una implementación basada en enumeraciones para asegurar thread-safety mientras gestiona el registro de prototipos de paletas. Por su parte, `ConexionSQLite.java` emplea double-checked locking junto con HikariCP para mantener un pool único de conexiones a la base de datos, evitando la sobrecarga de múltiples conexiones simultáneas y garantizando un acceso eficiente a la persistencia.

El patrón **Builder** facilita la construcción paso a paso de niveles con configuraciones complejas a través de `BuilderNivel.java`, orquestado por `DirectorNiveles.java` y apoyado por componentes auxiliares como `ConstructorMapa.java` y la enumeración `TipoBloque.java`. Esta estructura permite crear niveles con diferentes disposiciones de bloques, configuraciones de paletas y parámetros de dificultad de manera fluida y legible, evitando constructores con múltiples parámetros y facilitando la creación de variaciones de niveles.

El patrón **Factory** se aplica en dos contextos especializados. La factoría de inteligencia artificial, implementada en `FabricaIA.java`, crea instancias de `ServicioIA.java` configuradas según diferentes niveles de

dificultad definidos en la enumeración `DificultadIA.java`, utilizando `ConfiguracionDificultad.java` y `RepositorioConfiguracionesIA.java` para parametrizar el comportamiento. Esto permite abstraer la complejidad de instanciación de componentes de IA con diferentes características. Complementariamente, `FabricaItems.java` centraliza la creación de power-ups, encapsulando la lógica de instanciación y permitiendo agregar nuevos tipos de items sin modificar el código cliente.

El patrón **Memento** preserva y restaura estados de objetos sin violar su encapsulación, implementado en `MementoPaletas.java` para capturar configuraciones de paletas. Su aplicación en el editor de niveles permite implementar funcionalidad de deshacer cambios, mejorando significativamente la experiencia de usuario al crear o modificar niveles personalizados.

El patrón **Adapter** proporciona una interfaz unificada para diferentes esquemas de entrada mediante `AdaptadorEntrada.java`, implementado concretamente por `AdaptadorEntradaTeclado.java`. Esta abstracción facilita la integración de nuevos dispositivos de entrada en el futuro sin afectar el código del juego, manteniendo un punto de extensión flexible para soportar controles alternativos.

El patrón **Composite** estructura jerárquicamente los objetos del juego a través de la clase abstracta `ObjetoJuego.java` como componente base y `ObjetoJuegoCompuesto.java` como contenedor, con hojas específicas representadas por `Paleta.java`, `Pelota.java`, `Bloque.java` y `Nivel.java`. Esta organización permite tratar objetos individuales y composiciones de manera uniforme, simplificando operaciones como actualización y renderizado que se aplican recursivamente sobre estructuras complejas.

El patrón **Data Transfer Object (DTO)** desacopla la representación de datos entre capas mediante `NivelDTO.java` y `BloqueDTO.java`, permitiendo transferir información de niveles y bloques entre la capa de persistencia y el dominio sin exponer detalles de implementación de la base de datos ni acoplar las entidades del modelo directamente con la estructura de tablas.

El patrón **Repository** abstrae el acceso a datos a través de `RepositorioNiveles.java`, proporcionando una interfaz orientada a colecciones para operaciones de persistencia de niveles. Esta capa de abstracción aísla la lógica de negocio de los detalles específicos de SQLite, facilitando futuros cambios en la tecnología de almacenamiento.

El patrón **Prototype** permite la clonación de instancias predefinidas de paletas mediante el método `clonarPaleta()` en `GestorPrototiposPaleta.java` y el método `clonar()` en `Paleta.java`. Este mecanismo resulta más eficiente que la creación desde cero cuando se requieren múltiples instancias con configuraciones similares, especialmente al inicializar partidas con configuraciones predeterminadas.

El patrón **Facade** simplifica subsistemas complejos mediante interfaces unificadas en tres componentes clave. `CargadorRecursos.java` consolida la carga de diversos tipos de recursos como fuentes, archivos FXML, hojas de estilo CSS, videos, audio e imágenes, ocultando la complejidad de rutas y manejo de streams. `ServicioPersistencia.java` proporciona una interfaz simplificada para operaciones de persistencia, abstrayendo los detalles del repositorio y la conexión a base de datos. `GestorEscenas.java` centraliza la gestión de transiciones entre diferentes pantallas de la aplicación, simplificando la navegación y el ciclo de vida de las vistas.

Finalmente, el patrón **Value Object** garantiza inmutabilidad y thread-safety mediante Java Records en `ConfigPelota.java` y `ConfigPaleta.java`, encapsulando parámetros de configuración como valores inmutables que pueden compartirse sin riesgo de modificaciones concurrentes, mejorando la seguridad y predictibilidad del código en un contexto de animación con múltiples actualizaciones por segundo.

Estructura del Proyecto

El proyecto Pong Evolved se organiza siguiendo la estructura estándar de Maven, con la totalidad del código fuente ubicado bajo el directorio `src/main/java` y los recursos de la aplicación en `src/main/resources`. La arquitectura del sistema se fundamenta en una clara separación de responsabilidades mediante paquetes especializados que encapsulan funcionalidades específicas.

El punto de entrada de la aplicación se encuentra en el paquete `app`, que contiene la clase `AplicacionPong` responsable de inicializar el entorno JavaFX y orquestar el arranque del sistema. Esta clase constituye el núcleo desde el cual se despliegan todas las demás componentes de la aplicación.

El paquete `mvc` representa la columna vertebral arquitectónica del proyecto, implementando el patrón Model-View-Controller en su totalidad. Dentro de este paquete se distinguen tres subdivisiones principales: el paquete `modelo` alberga la lógica de negocio y el estado del juego a través de la clase `ModeloJuego`, junto con el subpaquete `entidades` que contiene las abstracciones de los elementos del juego. Este último se subdivide a su vez en `pelota` y `paleta`, cada uno conteniendo tanto la clase de entidad como su respectiva clase de configuración, mientras que en el nivel superior del paquete de entidades residen las clases `Bloque`, `Nivel`, `ObjetoJuego` y `ObjetoJuegoCompuesto`. Adicionalmente, el paquete `modelo` incluye el subpaquete `items` que define los power-ups del juego mediante las clases `ItemAumentoVelocidad`, `ItemNeblina` e `ItemRedimensionarPaleta`, así como el subpaquete `enums` que proporciona tipos enumerados para direcciones, modos de juego, lados horizontales y verticales, y tipos de nivel. El paquete `vista` agrupa las clases de presentación visual del sistema, incluyendo `VistaMenu`, `VistaJuego`, `VistaEditor`, `VistaSeleccionDificultad`, `VistaSeleccionNiveles` y el componente transversal `GestorEscenas` que coordina las transiciones entre diferentes escenas de la aplicación. Por su parte, el paquete `controlador` contiene la clase base `ControladorBase` y sus especializaciones `ControladorMenu`, `ControladorJuego`, `ControladorEditor`, `ControladorSeleccionDificultad`, `ControladorSeleccionNiveles`, así como controladores para diálogos específicos como `ControladorDialogoCargarNivel` y `ControladorDialogoSeleccionPorDificultad`.

El paquete `patrones` encapsula las implementaciones concretas de los nueve patrones de diseño empleados en el sistema. El subpaquete `strategy` se divide en dos categorías: `movimiento`, que contiene las estrategias de desplazamiento representadas por `EstrategiaMovimiento`, `EstrategiaMovimientoJugador` y `EstrategiaMovimientoIA`, esta última apoyada por las clases utilitarias `GeneradorErrorMovimiento`, `TemporizadorReaccion` y `CalcularTrayectoria`; y `colision`, que agrupa las estrategias de detección de colisiones a través de `EstrategiaColision`, `EstrategiaColisionPelotaPaleta`, `EstrategiaColisionPelotaBloque`, `EstrategiaColisionPelotaPared` y el orquestador `GestorColisiones`. El subpaquete `observer` implementa el patrón observador mediante `ObservadorJuego`, `GestorAudio` y `ObservadorUI`. El subpaquete `singleton` provee acceso a instancias únicas a través de `ConfiguracionGlobal` y `GestorPrototiposPaleta`. El subpaquete `builder` facilita la construcción de objetos complejos mediante `BuilderNivel`, `DirectorNiveles`, `ConstructorMapa` y la enumeración `TipoBloque`. El subpaquete `factory` se especializa en la creación de objetos, subdividiéndose en `ia` con `FabricaIA`, `ServicioIA`, `DificultadIA`, `ConfiguracionDificultad` y `RepositorioConfiguracionesIA`, e `items` con `FabricaItems`. El subpaquete `memento` implementa la funcionalidad de captura y restauración de estado mediante `MementoPaletas`, mientras que el subpaquete `adapter` proporciona abstracción de entrada a través de `AdaptadorEntrada` y `AdaptadorEntradaTeclado`.

El paquete `persistencia` gestiona el almacenamiento y recuperación de datos del sistema. Se estructura en tres subpaquetes: `conexion` que administra la conectividad con la base de datos SQLite mediante `ConexionSQLite`, `dto` que define los objetos de transferencia de datos `NivelDTO` y `BloqueDTO`, y `repositorio` que implementa el patrón repositorio a través de `RepositorioNiveles`. La clase `ServicioPersistencia` actúa como fachada que coordina las operaciones de persistencia del sistema.

El paquete `util` agrupa componentes utilitarios transversales al sistema, incluyendo `CargadorRecursos` para la carga de fuentes, imágenes y archivos FXML, `GestorVideos` para la reproducción de contenido multimedia, `RenderizadorJuego` para el dibujado de elementos visuales del juego, y `ParticleEmitter` que implementa el sistema de partículas para efectos visuales.

El directorio `src/main/resources` contiene todos los recursos no compilables de la aplicación, organiza-

dos en subdirectorios especializados: `fxml` almacena las definiciones de interfaz de usuario en formato FXML, `css` contiene las hojas de estilo para la presentación visual, `fuentes` incluye la tipografía personalizada Press Start 2P que confiere la estética retro al juego, `videos` alberga las previsualizaciones multimedia de los modos de juego, `imagenes` agrupa los recursos gráficos estáticos, y `audio` contiene los efectos sonoros y pistas musicales del sistema.

Tecnologías y Bibliotecas Utilizadas

El proyecto se fundamenta en JavaFX versión 17.0.12 como framework principal para la construcción de la interfaz gráfica de usuario, aprovechando sus cuatro módulos esenciales para distintos aspectos de la aplicación. El módulo `javafx.controls` proporciona los componentes visuales fundamentales como botones, etiquetas y paneles, mientras que `javafx.fxml` permite la definición declarativa de interfaces mediante archivos de marcado XML que separan la estructura visual de la lógica de control. El módulo `javafx.media` habilita la reproducción de contenido audiovisual, facilitando la integración de videos de presentación y efectos sonoros que enriquecen la experiencia interactiva, en tanto que `javafx.graphics` ofrece las capacidades de renderizado bidimensional necesarias para el sistema de partículas y la representación visual de entidades del juego. La gestión de dependencias y el proceso de construcción automatizado se realiza mediante Apache Maven, herramienta que orquesta la compilación del código fuente, la resolución de bibliotecas transitivas, el empaquetado de artefactos y la ejecución de la aplicación a través de plugins especializados como el Maven Compiler Plugin versión 3.11.0 para compilación con Java 17, el JavaFX Maven Plugin versión 0.0.8 para la integración del runtime de JavaFX, y el Maven JAR Plugin versión 3.3.0 para la generación de archivos ejecutables.

La arquitectura del proyecto adopta el paradigma de programación funcional pura mediante la biblioteca Vavr versión 0.10.4, la cual extiende las capacidades funcionales nativas de Java proporcionando estructuras de datos inmutables y persistentes que garantizan thread-safety sin necesidad de sincronización explícita. Esta biblioteca introduce tipos como `Option` para el manejo seguro de valores opcionales evitando referencias nulas, `Either` para representar computaciones que pueden resultar en éxito o fallo sin recurrir a excepciones, `Try` para encapsular operaciones que pueden lanzar excepciones transformándolas en valores manejables funcionalmente, y colecciones inmutables como `List`, `Set` y `Map` que facilitan transformaciones de datos mediante operaciones de orden superior. La adopción de Vavr permite expresar la lógica del dominio de manera declarativa y composicional, reduciendo la complejidad accidental asociada con mutabilidad y efectos secundarios, mientras se mantiene compatibilidad con el ecosistema estándar de Java a través de métodos de conversión entre colecciones nativas y funcionales.

El subsistema de persistencia se implementa mediante SQLite JDBC Driver versión 3.47.2.0, motor de base de datos embebido que proporciona almacenamiento relacional sin requerir procesos servidor externos, resultando ideal para aplicaciones de escritorio que necesitan persistencia local sin complejidad administrativa. La gestión eficiente de conexiones a la base de datos se optimiza mediante HikariCP versión 5.1.0, pool de conexiones de alto rendimiento que minimiza la latencia de acceso a datos mediante reutilización de conexiones, implementación de prepared statement caching y configuración adaptativa de parámetros según la carga del sistema. El sistema de registro y diagnóstico se fundamenta en Logback Classic versión 1.2.13, implementación de referencia de la interfaz SLF4J que proporciona capacidades avanzadas de logging incluyendo niveles configurables de severidad, rotación automática de archivos, filtrado condicional de mensajes y formateo personalizable de salidas, siendo particularmente necesario para registrar las operaciones del pool de conexiones HikariCP durante desarrollo y diagnóstico de problemas. La interfaz visual se enriquece mediante la biblioteca Ikonli versión 12.3.1 con su paquete de iconos FontAwesome 5, proporcionando iconografía vectorial escalable que mantiene calidad visual en diferentes resoluciones de pantalla y permite la aplicación de estilos CSS para integración consistente con la estética retro del juego, facilitando la construcción de interfaces expresivas mediante el módulo `ikonli-javafx` que integra transparentemente estos recursos con los componentes nativos de JavaFX.

Requisitos del Sistema

La ejecución satisfactoria del proyecto requiere un entorno configurado con Java Development Kit versión 17, la cual constituye la versión objetivo tanto para compilación como para ejecución del código fuente. La aplicación ha sido diseñada para operar en sistemas operativos Windows y Linux, garantizando compati-

bilidad multiplataforma mediante el uso de tecnologías independientes del sistema operativo subyacente. La gestión de dependencias y el proceso de construcción del proyecto se realizan mediante Apache Maven, requiriéndose una instalación funcional de Maven versión 3.0 o superior para la compilación y empaquetado de la aplicación.

El framework de interfaz gráfica JavaFX en su versión 17.0.12 constituye una dependencia fundamental del sistema, proporcionando los componentes necesarios para el renderizado de la interfaz visual, la reproducción de contenido multimedia y el manejo de eventos de usuario. Esta biblioteca se integra mediante los módulos `javafx.controls`, `javafx.fxml`, `javafx.media` y `javafx.graphics`, los cuales son gestionados automáticamente por Maven durante el proceso de construcción. Adicionalmente, el proyecto incorpora la biblioteca Vavr 0.10.4 para facilitar la programación funcional en Java, proporcionando estructuras de datos inmutables y abstracciones funcionales que mejoran la expresividad del código.

La persistencia de datos se implementa mediante SQLite JDBC Driver versión 3.47.2.0, el cual permite la interacción con bases de datos SQLite embebidas sin requerir instalación de servidores de base de datos externos. El sistema de gestión de conexiones HikariCP versión 5.1.0 optimiza el acceso a la base de datos mediante un pool de conexiones de alto rendimiento, mientras que Logback Classic 1.2.13 proporciona capacidades de registro y diagnóstico compatibles con la interfaz SLF4J. Para el enriquecimiento visual de la interfaz, se utiliza la biblioteca Ikonli versión 12.3.1 con el paquete de iconos FontAwesome 5, permitiendo la integración de iconografía vectorial escalable en los componentes JavaFX.

En términos de recursos de hardware, se recomienda un mínimo de 2 gigabytes de memoria RAM disponible para la máquina virtual Java, aunque para una experiencia óptima se sugieren 4 gigabytes o más, especialmente durante la ejecución de niveles con múltiples elementos gráficos y efectos de partículas. El espacio en disco requerido asciende aproximadamente a 300 megabytes, considerando la aplicación compilada, las dependencias de Maven, los recursos multimedia embebidos y el archivo de base de datos SQLite. Todos los archivos fuente utilizan codificación UTF-8, garantizando compatibilidad con caracteres especiales y facilitando la colaboración en entornos internacionales.

Instrucciones de Compilación y Ejecución

El proyecto Pong Evolved utiliza Apache Maven como sistema de construcción y gestión de dependencias, proporcionando un flujo de trabajo unificado para la compilación, el empaquetado y la ejecución de la aplicación. La estructura del proyecto sigue las convenciones estándar de Maven, con el código fuente ubicado en `src/main/java` y los recursos en `src/main/resources`, mientras que la configuración del proyecto se especifica en el archivo `pom.xml` ubicado en el directorio raíz del proyecto. Los comandos de Maven presentados a continuación son idénticos tanto para sistemas operativos Windows como para distribuciones Linux, garantizando una experiencia consistente de desarrollo multiplataforma.

Compilación del Proyecto

Para compilar el código fuente de la aplicación junto con todas sus dependencias, es necesario ejecutar el siguiente comando desde el directorio raíz del proyecto:

```
mvn clean compile
```

Este comando ejecuta dos fases del ciclo de vida de Maven de manera secuencial. La fase `clean` elimina el contenido del directorio `target`, removiendo todos los artefactos generados en compilaciones previas y garantizando una construcción limpia libre de residuos de versiones anteriores. Posteriormente, la fase `compile` procesa el código fuente ubicado en `src/main/java`, invocando el compilador de Java especificado en el plugin Maven Compiler (configurado para Java 17), verificando la sintaxis del código, resolviendo las dependencias declaradas en el archivo `pom.xml`, y generando los archivos `.class` correspondientes en el directorio `target/classes`. Durante este proceso, Maven descarga automáticamente todas las bibliotecas necesarias desde los repositorios centrales si no se encuentran previamente en el repositorio local, incluyendo JavaFX, Vavr, SQLite JDBC, HikariCP, Logback e Ikonli junto con sus dependencias transitivas.

Ejecución de la Aplicación

Una vez compilado el proyecto satisfactoriamente, la aplicación puede ejecutarse mediante el siguiente comando:

```
mvn javafx:run
```

Este comando invoca el plugin JavaFX Maven Plugin, el cual configura el classpath necesario incluyendo todos los módulos de JavaFX requeridos, establece las opciones de la máquina virtual Java apropiadas para la ejecución de aplicaciones JavaFX, y lanza la clase principal `app.AplicacionPong` especificada en la configuración del plugin. El framework JavaFX inicializa la interfaz gráfica, carga los recursos necesarios desde `src/main/resources`, y despliega la ventana principal de la aplicación mostrando el menú inicial del juego. Es importante destacar que este comando no requiere la generación previa de un archivo JAR ejecutable, facilitando ciclos rápidos de desarrollo y pruebas durante la fase de implementación.

Empaquetado de la Aplicación

Para generar un archivo JAR ejecutable que encapsule la aplicación compilada junto con su manifiesto, puede utilizarse el siguiente comando:

```
mvn package
```

La fase `package` ejecuta automáticamente todas las fases previas del ciclo de vida de Maven, incluyendo la validación del proyecto, la compilación del código fuente y la ejecución de pruebas unitarias si están presentes. Posteriormente, el Maven JAR Plugin empaqueta las clases compiladas ubicadas en `target/classes` dentro de un archivo JAR que se genera en el directorio `target` con el nombre especificado en el elemento `finalName` del archivo `pom.xml`. El archivo JAR resultante incluye un manifiesto que especifica la clase principal de la aplicación, facilitando su distribución y despliegue en entornos de producción.

Consideraciones Adicionales

La aplicación requiere acceso a un sistema de persistencia basado en SQLite para el almacenamiento de niveles personalizados creados mediante el editor integrado. Al ejecutarse por primera vez, el sistema verifica la existencia del directorio `data` en la raíz del proyecto y, de no encontrarlo, lo crea automáticamente. Dentro de este directorio se genera el archivo de base de datos `pong.db` con el esquema necesario para almacenar la información de niveles y bloques. Las operaciones de creación y migración del esquema se ejecutan de manera transparente durante el arranque de la aplicación, por lo que no se requiere intervención manual para la inicialización de la base de datos.

En el caso de que el proyecto utilice características de serialización de Java que generen advertencias del compilador o de la máquina virtual Java, es posible suprimir estos avisos mediante la anotación `@SuppressWarnings("unchecked")` en el código fuente, o alternativamente, deshabilitando las verificaciones de serialización al invocar la JVM con el argumento `-Xlint:-serial`. Sin embargo, la implementación actual de Pong Evolved no emplea mecanismos de serialización nativa de Java, optando en su lugar por el uso de objetos de transferencia de datos y consultas SQL para la persistencia, por lo que estas consideraciones no resultan aplicables al presente proyecto.

Todos los comandos de Maven presentados en esta sección deben ejecutarse desde el directorio raíz del proyecto, donde reside el archivo `pom.xml`, utilizando una terminal de comandos o símbolo del sistema con acceso a una instalación funcional de Maven agregada al `PATH` del sistema operativo. La salida de los comandos proporciona información detallada sobre el progreso de cada fase del ciclo de vida, incluyendo la resolución de dependencias, el proceso de compilación y cualquier advertencia o error que pueda surgir durante la construcción del proyecto.

Instrucciones de Ejecución con Docker

El proyecto Pong Evolved incorpora soporte para despliegue mediante contenedores Docker, proporcionando un entorno de ejecución estandarizado que encapsula todas las dependencias necesarias para la aplicación, incluyendo el entorno de ejecución de Java, las bibliotecas JavaFX y el servidor VNC para acceso remoto a la interfaz gráfica. Esta aproximación basada en contenedores garantiza consistencia en el despliegue a través de diferentes sistemas operativos y configuraciones de hardware, simplificando significativamente el proceso de instalación y eliminando problemas de incompatibilidad de versiones de dependencias. El contenedor configura automáticamente un servidor web ligero que sirve una interfaz VNC accesible mediante navegador, permitiendo la interacción con la aplicación gráfica sin necesidad de instalar clientes VNC adicionales.

Construcción de la Imagen

El primer paso para ejecutar la aplicación mediante Docker consiste en construir la imagen del contenedor a partir del archivo `Dockerfile` ubicado en el directorio raíz del proyecto. Este proceso debe ejecutarse una única vez, o cuando se realicen modificaciones al código fuente que requieran regenerar la imagen. El siguiente comando construye la imagen y la etiqueta con el identificador `pong-evolved`:

```
docker build -t pong-evolved .
```

Durante el proceso de construcción, Docker ejecuta las instrucciones especificadas en el `Dockerfile`, lo cual incluye la descarga de la imagen base, la instalación de las dependencias del sistema operativo necesarias para JavaFX y VNC, la copia del código fuente de la aplicación al sistema de archivos del contenedor, la compilación del proyecto mediante Maven, y la configuración del servidor VNC junto con el servidor web noVNC que proporciona acceso mediante navegador. Este proceso puede tomar varios minutos en su primera ejecución debido a la descarga de imágenes base y dependencias, aunque ejecuciones posteriores serán significativamente más rápidas gracias al sistema de caché de capas de Docker.

Ejecución del Contenedor

Una vez construida la imagen, el contenedor puede iniciarse mediante dos modalidades de ejecución que difieren en el tratamiento de la persistencia de datos. La primera opción ejecuta el contenedor sin persistencia explícita de la base de datos, lo cual implica que los niveles personalizados creados mediante el editor se perderán cuando el contenedor sea detenido o eliminado. Esta modalidad resulta apropiada para pruebas temporales o demostraciones que no requieren conservar datos entre sesiones:

```
docker run -p 80:80 -p 5901:5901 pong-evolved
```

La segunda opción, recomendada para uso regular, monta un volumen de Docker que persiste el directorio `/app/data` del contenedor, donde se almacena el archivo de base de datos SQLite. Esta configuración garantiza que los niveles personalizados creados permanezcan disponibles entre ejecuciones del contenedor, sobreviviendo incluso a la eliminación y recreación del mismo:

```
docker run -p 80:80 -p 5901:5901 -v pong-data:/app/data pong-evolved
```

Ambos comandos exponen dos puertos del contenedor hacia el sistema host: el puerto 80 proporciona acceso a la interfaz web de noVNC que permite controlar la aplicación desde cualquier navegador web moderno, mientras que el puerto 5901 expone el servidor VNC nativo para conexión mediante clientes VNC tradicionales. El parámetro `-v pong-data:/app/data` en la segunda opción crea un volumen nombrado gestionado por Docker que persiste independientemente del ciclo de vida del contenedor.

Acceso a la Aplicación

Una vez iniciado el contenedor, la aplicación se encuentra accesible a través de dos métodos distintos. El método recomendado y más conveniente consiste en utilizar un navegador web para acceder a la interfaz

noVNC dirigiéndose a la dirección `http://localhost`. Esta aproximación funciona sin necesidad de software adicional en sistemas Windows, Linux y MacOS, proporcionando una experiencia de usuario consistente independientemente del sistema operativo subyacente. La interfaz web despliega el escritorio virtual del contenedor donde se ejecuta la aplicación Pong Evolved, permitiendo la interacción mediante mouse y teclado directamente desde el navegador.

Alternativamente, usuarios que prefieran utilizar clientes VNC nativos pueden conectarse al servidor VNC expuesto en el puerto 5901 mediante la dirección `localhost:5901`. Esta opción puede proporcionar mejor rendimiento de renderizado en redes locales rápidas, aunque requiere la instalación previa de software cliente VNC como TigerVNC, RealVNC o TightVNC. Ambos métodos de acceso proporcionan control completo sobre la aplicación, incluyendo la capacidad de navegar por los menús, iniciar partidas, configurar opciones y utilizar el editor de niveles.

Detención del Contenedor

Para finalizar la ejecución del contenedor y liberar los recursos del sistema, es necesario interrumpir el proceso de Docker que se encuentra ejecutándose en primer plano. En la terminal donde se lanzó el comando `docker run`, debe presionarse la combinación de teclas `Ctrl+C` de manera repetida hasta que el contenedor se detenga completamente y el control retorne al prompt de comandos. En algunos casos puede ser necesario presionar esta combinación varias veces para forzar la terminación del servidor VNC y todos los procesos asociados dentro del contenedor. Una vez detenido el contenedor, los puertos 80 y 5901 quedan liberados y disponibles para otros servicios del sistema.

Guía de Usuario

El sistema Pong Evolved proporciona una experiencia de juego interactiva que se despliega inmediatamente tras la ejecución de la aplicación, presentando al usuario una interfaz de menú principal desde la cual se accede a las diferentes modalidades de juego disponibles. La navegación a través del sistema se realiza mediante interacciones con elementos visuales diseñados siguiendo una estética retro que evoca la experiencia original del juego clásico Pong, complementada con mejoras modernas tanto en la presentación gráfica como en las mecánicas de juego.

La aplicación ofrece tres modalidades principales de interacción que satisfacen diferentes necesidades de entretenimiento y personalización. La primera modalidad, denominada modo de un jugador, permite al usuario enfrentarse contra un oponente controlado por inteligencia artificial cuyo comportamiento puede ajustarse a través de diez niveles de dificultad progresiva. El nivel más bajo, identificado como Nivel 1 Principiante, proporciona una experiencia accesible para usuarios novatos mediante una inteligencia artificial que exhibe tiempos de reacción considerablemente lentos y una precisión de movimiento significativamente reducida, facilitando que el jugador humano desarrolle familiaridad con las mecánicas básicas del juego. A medida que se incrementa el nivel de dificultad, la inteligencia artificial mejora tanto su velocidad de reacción como su capacidad predictiva del movimiento de la pelota, culminando en el Nivel 10 Leyenda, donde el oponente artificial despliega tiempos de reacción prácticamente instantáneos y una precisión casi perfecta en el posicionamiento de su paleta, representando un desafío significativo incluso para jugadores experimentados. Los niveles intermedios proporcionan una progresión gradual que abarca desde el Nivel 2 Novato y Nivel 3 Aprendiz con características de lentitud e imprecisión moderadas, pasando por el Nivel 4 Básico y Nivel 5 Intermedio con reacciones y precisión balanceadas, continuando con el Nivel 6 Competente y Nivel 7 Avanzado que ofrecen velocidad y precisión considerables, hasta alcanzar el Nivel 8 Experto y Nivel 9 Maestro que se aproximan a las capacidades máximas del sistema de inteligencia artificial.

La segunda modalidad disponible corresponde al modo de dos jugadores, diseñado para facilitar experiencias de juego competitivas en un contexto local donde dos usuarios humanos pueden enfrentarse directamente compartiendo el mismo dispositivo. Esta modalidad elimina la presencia de la inteligencia artificial y permite que ambos participantes controlen sus respectivas paletas de manera simultánea mediante esquemas de control diferenciados, proporcionando una experiencia de juego tradicional que preserva la esencia del Pong original mientras se beneficia de las mejoras gráficas y de la arquitectura de software moderna implementada en el sistema.

La tercera modalidad, conocida como modo constructor, proporciona al usuario capacidades de edición y personalización avanzadas mediante un editor visual de niveles que permite la creación de configuraciones personalizadas del campo de juego. Este editor facilita la colocación de bloques destructibles en disposiciones arbitrarias, la configuración de parámetros de las paletas y la pelota, y el ajuste de diversos aspectos del comportamiento del juego. Los niveles creados mediante esta herramienta pueden persistirse en la base de datos SQLite del sistema, permitiendo su recuperación y utilización en sesiones futuras. Esta funcionalidad de persistencia garantiza que el esfuerzo invertido en el diseño de configuraciones personalizadas no se pierde al finalizar la aplicación, facilitando la construcción iterativa de niveles complejos y la compartición de configuraciones entre diferentes instancias de la aplicación.

El esquema de control del sistema se fundamenta en el uso del teclado como dispositivo de entrada principal, proporcionando combinaciones de teclas específicas para las diferentes acciones disponibles durante el juego. Durante el transcurso de una partida, el movimiento de la paleta correspondiente al primer jugador se controla mediante las teclas W y S, donde la primera produce desplazamiento hacia la porción superior del área de juego y la segunda genera movimiento hacia la región inferior. En el modo de dos jugadores, la paleta del segundo participante responde a las teclas de dirección del cursor, específicamente la flecha hacia arriba para movimiento ascendente y la flecha hacia abajo para desplazamiento descendente. Una vez que los jugadores se encuentran posicionados en la pantalla de juego y el sistema se encuentra en estado de espera para el inicio de la partida, la tecla de espacio activa el comienzo del movimiento de la pelota y la transición al estado activo del juego donde comienza el cómputo de puntuación. Durante el transcurso de cualquier partida, la tecla Alt proporciona funcionalidad de pausa que suspende temporalmente la actualización del estado del juego y el movimiento de todos los elementos dinámicos, permitiendo al usuario interrumpir la experiencia sin perder el progreso actual.

El sistema incorpora adicionalmente controles para la gestión del modo de visualización de la ventana de la aplicación. La tecla P permite alternar entre el modo de ventana convencional y el modo de pantalla completa, maximizando el área de visualización disponible y eliminando elementos de distracción de la interfaz del sistema operativo. La tecla Escape proporciona una función complementaria que permite salir del modo de pantalla completa cuando este se encuentra activo, retornando la aplicación a su presentación en ventana convencional. Estas opciones de visualización facilitan la adaptación de la experiencia a las preferencias individuales del usuario y a las características específicas del entorno de visualización disponible.

Funcionalidades Implementadas

El proyecto Pong Evolved incorpora un conjunto amplio de funcionalidades que trascienden la implementación básica del juego clásico, demostrando la aplicación práctica de principios avanzados de ingeniería de software y arquitectura de sistemas interactivos. Las características principales del sistema se fundamentan en tres modalidades de juego completamente funcionales que proporcionan experiencias diferenciadas según las preferencias del usuario. La modalidad de un jugador implementa un sistema de inteligencia artificial adaptativo con diez niveles de dificultad progresiva, cada uno parametrizado mediante configuraciones específicas de tiempo de reacción, precisión de movimiento y capacidad predictiva, permitiendo que tanto usuarios novatos como experimentados encuentren un nivel de desafío apropiado a sus habilidades. La modalidad de dos jugadores facilita experiencias competitivas locales mediante esquemas de control diferenciados que permiten la participación simultánea de dos usuarios humanos compartiendo el mismo dispositivo, preservando la esencia del juego original mientras se beneficia de las mejoras técnicas implementadas en la arquitectura del sistema. La modalidad de edición proporciona un entorno visual interactivo para la creación y personalización de configuraciones de juego, permitiendo el posicionamiento arbitrario de bloques destructibles con diferentes características de resistencia y tipo, la configuración de parámetros de entidades del juego, y la persistencia de estas configuraciones para su utilización en sesiones futuras.

El sistema de puntuación y temporización implementa mecánicas de juego dinámicas que registran el progreso de los participantes a lo largo de partidas con duración limitada, notificando eventos relevantes a través del patrón Observer para mantener sincronizada la interfaz visual con el estado interno del modelo. Los bloques destructibles incorporan propiedades de resistencia variable que determinan el número de impactos necesarios para su eliminación, junto con un sistema de tipificación que permite definir diferentes comportamientos visuales y funcionales. El subsistema de power-ups proporciona elementos coleccionables que modifican temporalmente las propiedades del juego, incluyendo items de aumento de velocidad de la pelota que intensifican la dificultad del juego, items de efecto niebla que reducen la visibilidad del campo de juego añadiendo un desafío visual, e items de redimensionamiento de paleta que alteran las dimensiones de los controles del jugador afectando la facilidad de interceptación de la pelota. El sistema de efectos visuales implementa un emisor de partículas configurable que genera animaciones procedurales para representar colisiones, destrucción de bloques y otros eventos significativos del juego, enriqueciendo la retroalimentación visual proporcionada al usuario durante la interacción.

Más allá de las características fundamentales del juego, el proyecto incorpora elementos técnicos adicionales que demuestran competencia en el manejo de tecnologías modernas de desarrollo de aplicaciones de escritorio. La interfaz gráfica se construye íntegramente mediante JavaFX, aprovechando su arquitectura de grafos de escena para el renderizado eficiente de elementos visuales y su modelo de eventos para la gestión de interacciones del usuario. El sistema comprende múltiples vistas especializadas incluyendo el menú principal, la vista del juego activo, el editor de niveles, los diálogos de selección de dificultad y nivel, cada una definida mediante archivos FXML que separan la estructura visual de la lógica de control, facilitando el mantenimiento independiente de ambos aspectos. La presentación visual adopta una estética retro mediante hojas de estilo CSS personalizadas que definen colores, espaciados y efectos visuales inspirados en la apariencia original del juego Pong, complementada con la fuente tipográfica pixelada Press Start 2P que refuerza la ambientación nostálgica del sistema. La iconografía vectorial proporcionada por la biblioteca Ikonli con el conjunto de iconos FontAwesome 5 permite la integración de símbolos escalables que mantienen calidad visual independientemente de la resolución de pantalla, aplicándose estilos CSS consistentes con el resto de la interfaz.

El subsistema de persistencia implementa almacenamiento relacional mediante SQLite, motor de base de datos embebido que proporciona capacidades SQL completas sin requerir procesos servidor externos, resultando apropiado para aplicaciones de escritorio que necesitan persistencia local sin complejidad administrativa. La gestión de conexiones se optimiza mediante HikariCP, pool de conexiones de alto rendimiento que minimiza la latencia de acceso a datos mediante reutilización de conexiones establecidas, implementación de caché de prepared statements y configuración adaptativa de parámetros de pool según la carga del sistema. Esta arquitectura de acceso a datos facilita operaciones concurrentes seguras sobre la base de datos mientras mantiene un consumo eficiente de recursos del sistema. El subsistema de threading implementa el bucle principal del juego mediante AnimationTimer de JavaFX, mecanismo que sincroniza las actualizaciones del estado del juego con el ciclo de refresco de la interfaz gráfica,

garantizando animaciones fluidas con frecuencias de actualización consistentes. Este enfoque aprovecha el framework de concurrencia de JavaFX que gestiona automáticamente la coordinación entre el hilo de la interfaz de usuario y los hilos de procesamiento en segundo plano, previniendo condiciones de carrera y garantizando la consistencia visual del sistema.

El subsistema de audio proporciona retroalimentación sonora inmersiva mediante dos categorías diferenciadas de contenido auditivo. Los efectos de sonido se implementan mediante AudioClip, clase optimizada para la reproducción repetida de clips de audio cortos con latencia mínima, apropiada para sonidos de colisión, destrucción de bloques y activación de power-ups que requieren respuesta inmediata a eventos del juego. La música de fondo se gestiona mediante MediaPlayer configurado con reproducción en bucle infinito, proporcionando una banda sonora continua que acompaña la experiencia de juego sin interrupciones perceptibles. El volumen de ambas categorías puede ajustarse de manera independiente, permitiendo que los usuarios personalicen el balance entre efectos y música según sus preferencias. El subsistema de video implementa capacidades de reproducción de contenido multimedia mediante MediaPlayer, utilizado específicamente para mostrar previsualizaciones animadas de las diferentes modalidades de juego en el menú principal. Estas previsualizaciones proporcionan retroalimentación visual que ilustra las características de cada modo antes de que el usuario realice su selección, mejorando la comprensibilidad de las opciones disponibles y enriqueciendo la presentación general del sistema.

Base de Datos

El sistema de persistencia de Pong Evolved se fundamenta en una base de datos SQLite almacenada en el archivo `pong.db` ubicado dentro del directorio `data` en la raíz del proyecto. Esta ubicación se determina automáticamente durante la inicialización del sistema, el cual verifica la existencia del directorio de datos y lo crea si resulta necesario antes de establecer la conexión con el motor de base de datos. La naturaleza embebida de SQLite elimina la necesidad de procesos servidor independientes y simplifica significativamente el despliegue de la aplicación, dado que el archivo de base de datos constituye un único fichero autónomo que puede copiarse, respaldarse o distribuirse sin requerir configuración adicional. El acceso a este archivo se gestiona mediante HikariCP, sistema de pooling de conexiones que mantiene un conjunto de conexiones reutilizables configuradas para optimizar el rendimiento de operaciones de lectura y escritura concurrentes mientras minimiza la sobrecarga asociada con el establecimiento repetido de conexiones.

La estructura del esquema de base de datos se compone de dos tablas relacionales que modelan los conceptos fundamentales del dominio de niveles personalizados. La tabla `niveles` constituye la entidad principal que almacena la información descriptiva y los metadatos de cada configuración de nivel creada mediante el editor. Esta tabla define un identificador textual que sirve como clave primaria garantizando la unicidad de cada nivel en el sistema, un campo de nombre que proporciona una etiqueta legible por humanos para facilitar la identificación de niveles en las interfaces de selección, un campo de creador opcional que permite registrar la autoría de configuraciones personalizadas facilitando el reconocimiento de contribuciones individuales, un campo numérico de dificultad que clasifica los niveles según su complejidad percibida permitiendo filtrados y ordenamientos basados en este criterio, un indicador booleano almacenado como entero que distingue entre niveles creados por usuarios y niveles predefinidos del sistema, y una marca temporal textual que registra el momento de creación del nivel proporcionando información cronológica útil para ordenamientos temporales y auditoría de modificaciones.

La tabla `bloques` complementa la información de niveles almacenando las características individuales de cada elemento destructible que compone la disposición espacial del campo de juego. Esta tabla implementa un identificador numérico autoincrementado que sirve como clave primaria garantizando la unicidad de cada registro de bloque, una referencia textual al nivel propietario que establece la relación entre bloques y niveles mediante una clave foránea con comportamiento de eliminación en cascada, dos campos de precisión real que codifican las coordenadas cartesianas de la posición del bloque en el espacio bidimensional del campo de juego, dos campos adicionales de precisión real que especifican las dimensiones geométricas del bloque determinando su extensión horizontal y vertical, un campo numérico entero que representa la resistencia del bloque definiendo el número de impactos necesarios para su destrucción, y un campo textual que identifica el tipo del bloque permitiendo la diferenciación de comportamientos y representaciones visuales asociadas a distintas categorías de bloques. La relación

entre ambas tablas se establece mediante la clave foránea `nivel_id` en la tabla de bloques que referencia el identificador primario de la tabla de niveles, configurada con la semántica de eliminación en cascada que garantiza la integridad referencial del sistema eliminando automáticamente todos los bloques asociados cuando se elimina un nivel, previniendo la existencia de registros huérfanos que comprometerían la consistencia de los datos persistidos.

El esquema implementa las tablas mediante sentencias SQL con la cláusula `IF NOT EXISTS`, permitiendo que el código de inicialización se ejecute de manera idempotente múltiples veces sin generar errores cuando las tablas ya existen previamente. Esta aproximación facilita la evolución del esquema en versiones futuras mediante la adición de nuevas sentencias de creación o alteración que se ejecutan condicionalmente según el estado actual de la base de datos. Las transacciones se gestionan explícitamente mediante el método funcional `ejecutarEnTransaccion` que encapsula operaciones de escritura en bloques transaccionales con confirmación automática en caso de éxito o reversión automática ante excepciones, garantizando que las modificaciones al estado persistente se aplican de manera atómica sin dejar la base de datos en estados inconsistentes intermedios. Esta arquitectura de persistencia proporciona una base sólida para la funcionalidad del editor de niveles, permitiendo que las configuraciones creadas por los usuarios se almacenen de manera confiable y se recuperen en sesiones futuras con fidelidad completa respecto a su definición original.